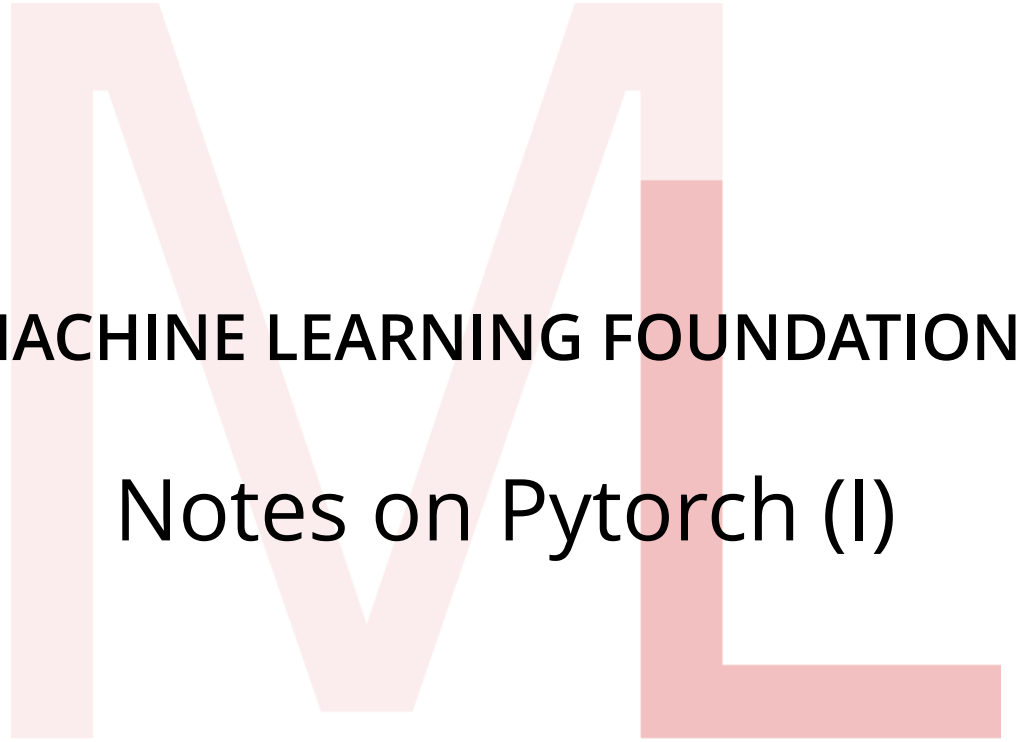
MACHINE LEARNING FOUNDATIONS

Notes on Pytorch (I)



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

PyTorch

- PyTorch was released in 2016 as a successor to Torch framework
- PyTorch has gained popularity due to its user-friendly design and dynamic computation graph capabilities
- Core features
  - Dynamic computation graphs
  - GPU acceleration support
  - Python integration
  - Ease of debugging

Developing ML models

- The easiest way of developing custom models (including layer, submodels, etc…) in PyTorch is by **inheriting from nn.Module**

```python
import torch
import torch.nn as nn

class CustomModel(nn.Module):
    def __init__(self):
        super(CustomModel, self).__init__()
        # Define layers here

    def forward(self, x):
        # Define forward pass here
        return x

# Instantiate the model
model = CustomModel()
```

Developing ML models

- When developing a custom model, we need to code the constructor and the forward method
  - __init__: we stablish possible input parameters to the model (for instance, the size of the input dimension) and we define and initialize all the (learnable) parameters of the model and any other variable (if needed)
  - Forward: we assume x as a set of input samples and we perform any transformation to x, returning the output of the model
- Once implemented,
  - We instantiate the CustomModel class
    - model = CustomModel()
  - We transform a set of input samples into outputs
    - y = model(x) -> means calling the forward method.

Developing ML models

- The flexibility of nn.Module allows us to define any
  - Layer
  - Set of layers
  - Submodel
  - Full model
  - …

  by inheriting from it.

Parameter specification

- In the __init__method, we can stablish any variable as a **learnable parameter** of the model using nn.Parameter.

```
class CustomModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.tensor = torch.randn(10)  # ❌ Not a parameter
        self.param = nn.Parameter(torch.randn(10))  # ✅ Trainable parameter

    def forward(self, x):
        return x
```

- This means that the optimizer will consider param as a learnable parameter, and will manage gradient, update, etc.

Training loop

- Once the model has been created, the next usual piece of code is the training loop, where we need to specify
  - Loss function
  - Optimizer
  - Update of parameters

Training loop

- Usual loss functions
  - nn.BCELoss(): binary cross entropy between target and output **probabilities**
  - nn.BCEWithLogitsLoss(): combines a sigmoid layer and the BCELoss in one class.
  - nn.CrossEntropyLoss(): cross entropy between input **logits** and target
  - nn.MSELoss(), nn.L1Loss(), ...
- It is important to correctly manage logits/probabilities, size of input/targets,

Training loop

- Optimizer
  - From now, we will use SGD
  - The parameters of the model can be set into the optimizer with
    - model.parameters()
  - Usual calculation and update of parameters
    - zero_grad()
    - backward()
    - step()
- If we need to work with the model,but we don’t need to calculate gradients, we specify the content
  - with torch.no_grad():
    - …